



```
import A5O
set_option autoImplicit false
```

```
/-! # A5O — Synthetic necessity.
```

Companion to A5O.lean v4 (SHA-256 0e5ebe3f...5b452), imported unmodified.

Closes the remainder of adversarial Finding A (review dated 2026-09-22):
 `achievesADA` contains `O` by definition, so `U3\_reduction` is analytic.
 Closure.lean's `Accountable` also contains `O`. Neither is the predicate the
 finding asks for.

This file defines accountability as a THIRD-PARTY OBSERVABLE that mentions
 none of C, I, M, S, R, Appointed, and proves that it entails v4's `O` at a
 derived instance. The six conjuncts are consequences, not stipulations.

Model of what a third party has, from public inputs and no operator
 testimony (theapertures.app's surface, abstracted):

```
resolve  — the public verifier's resolution of an execution to a
           ledger entry id (appointment_ref)
ledger   — the public grant ledger, by entry id
registered — the public registry of principals
withdrawn — the public withdrawal (revocation) set, by entry and time
```

Each grant carries the principal, the agent, the acts it NAMES (mandate)
 and the BOUND it states (scope ceiling) — the live gate's two distinct
 checkers — and the moment it was made.

Time := Nat throughout (as in Hops.lean).

ONE DEFINITIONAL COMMITMENT, stated here so it can be disputed in the
 open: "P authorized this act" means P's grant was MADE BEFORE the act and
 NOT WITHDRAWN AS OF the act. Ratification after the fact is not
 authorization. Nothing else in `ThirdPartyAccountable` is a reading of any
 conjunct. -/

```
namespace Synthetic
```

```
structure Grant (Principal Agent Action : Type) where
  principal : Principal
  agent     : Agent
  names     : Action → Prop
  bound     : Action → Prop
  made      : Nat
```

```

structure Observable (Principal Agent Action : Type) where
  resolve   : Agent → Action → Nat → Option Nat
  ledger    : Nat → Option (Grant Principal Agent Action)
  registered : Principal → Prop
  withdrawn : Nat → Nat → Prop

```

```

section Defs

```

```

variable {Principal Agent Action AoC : Type}
variable (obs : Observable Principal Agent Action)
variable (inAoC : Action → AoC → Prop)
variable (Executes : Agent → Action → Nat → Prop)

```

```

/-- Entry `g` (content `e`) has not been withdrawn at any moment after it
was made and at or before `t`. -/

```

```

def Standing (g : Nat) (e : Grant Principal Agent Action) (t : Nat) : Prop :=
  ¬ ∃ t'', e.made < t'' ∧ t'' ≤ t ∧ obs.withdrawn g t''

```

```

/-- A third party can determine, from public inputs alone, who authorized
`A` doing `X` at `t`. No conjunct of `O` is named. -/

```

```

def Authorized (A : Agent) (X : Action) (t : Nat) : Prop :=
  ∃ (g : Nat) (e : Grant Principal Agent Action),
    obs.resolve A X t = some g ∧
    obs.ledger g = some e ∧
    obs.registered e.principal ∧
    e.agent = A ∧
    e.names X ∧
    e.bound X ∧
    e.made < t ∧
    Standing obs g e t

```

```

/-- THE INDEPENDENT PREDICATE. Every execution in an area of consequence
is third-party determinable in the sense above. -/

```

```

def ThirdPartyAccountable : Prop :=
  ∀ (A : Agent) (X : Action) (t : Nat) (d : AoC),
    inAoC X d → Executes A X t → Authorized obs A X t

```

```

-- ---- The six conjuncts, DERIVED from the observable -----

```

```

/-- C: the public inputs bind the act to P's ledger entry. -/

```

```

def dC (P : Principal) (A : Agent) (X : Action) (t : Nat) : Prop :=
  ∃ (g : Nat) (e : Grant Principal Agent Action),
    obs.resolve A X t = some g ∧ obs.ledger g = some e ∧
    e.principal = P ∧ e.agent = A

```

-- I: P resolves in the public registry. -/

def dI (P : Principal) : Prop := obs.registered P

-- M: some grant for A names X. -/

def dM (A : Agent) (X : Action) : Prop :=
 ∃ (g : Nat) (e : Grant Principal Agent Action),
 obs.ledger g = some e ∧ e.agent = A ∧ e.names X

-- S: some grant for A bounds X. -/

def dS (A : Agent) (X : Action) : Prop :=
 ∃ (g : Nat) (e : Grant Principal Agent Action),
 obs.ledger g = some e ∧ e.agent = A ∧ e.bound X

-- R: whatever entry of P's the act resolves to is standing as of t. -/

def dR (P : Principal) (A : Agent) (X : Action) (t : Nat) : Prop :=
 ∀ (g : Nat) (e : Grant Principal Agent Action),
 obs.resolve A X t = some g → obs.ledger g = some e →
 e.principal = P → Standing obs g e t

-- Appointed: P's grant of A, naming and bounding X, occurred before t.

Does NOT mention `resolve` — separable from dC (see countermodels). -/

def dApp (P : Principal) (A : Agent) (X : Action) (t : Nat) : Prop :=
 ∃ (g : Nat) (e : Grant Principal Agent Action),
 obs.ledger g = some e ∧ e.principal = P ∧ e.agent = A ∧
 e.names X ∧ e.bound X ∧ e.made < t

-- v4's O at the derived instance. -/

abbrev Od (P : Principal) (A : Agent) (X : Action) (t : Nat) : Prop :=
 O (dC obs) (dI obs) (dM obs) (dS obs) (dR obs) (dApp obs) P A X t

-- ---- Main result -----

theorem authorized_implies_O (A : Agent) (X : Action) (t : Nat)
 (h : Authorized obs A X t) : ∃ P : Principal, Od obs P A X t := by
 obtain ⟨g, e, hres, hled, hreg, hag, hn, hb, hmade, hst⟩ := h
 refine ⟨e.principal,
 ⟨g, e, hres, hled, rfl, hag⟩,
 hreg,
 ⟨g, e, hled, hag, hn⟩,
 ⟨g, e, hled, hag, hb⟩,
 ?_,
 ⟨g, e, hled, rfl, hag, hn, hb, hmade⟩⟩
 intro g' e' hres' hled' _

```

have hg : g = g' := Option.some.inj (hres.symm.trans hres')
subst hg
have he : e = e' := Option.some.inj (hled.symm.trans hled')
subst he
exact hst

```

```

/-- SYNTHETIC NECESSITY. Third-party accountability, defined with no
reference to any conjunct, entails O on every consequential execution. -/
theorem synthetic_necessity (h : ThirdPartyAccountable obs inAoC Executes) :
  ∀ (A : Agent) (X : Action) (t : Nat) (d : AoC),
    inAoC X d → Executes A X t → ∃ P : Principal, Od obs P A X t :=
  fun A X t d hd he => authorized_implies_O obs A X t (h A X t d hd he)

```

```

/-- Single governor is a CONSEQUENCE: the resolver is a function of public
inputs, so it returns one entry, hence one principal. -/
theorem dC_functional : CFunctional (dC obs) := by
  intro P P' A X t hP hP'
  obtain ⟨g, e, hres, hled, hp, _⟩ := hP
  obtain ⟨g', e', hres', hled', hp', _⟩ := hP'
  have hg : g = g' := Option.some.inj (hres.symm.trans hres')
  subst hg
  have he : e = e' := Option.some.inj (hled.symm.trans hled')
  subst he
  exact hp.symm.trans hp'

```

```

theorem single_governor :
  ∀ (P P' : Principal) (A : Agent) (X : Action) (t : Nat),
    Od obs P A X t → Od obs P' A X t → P = P' :=
  O_unique (dC obs) (dl obs) (dM obs) (dS obs) (dR obs) (dApp obs)
  (dC_functional obs)

```

```

/-- Revocation dominance is a CONSEQUENCE: if the entry an execution would
resolve to has been withdrawn as of t, an accountable system did not let
it execute. -/

```

```

theorem revocation_dominates (h : ThirdPartyAccountable obs inAoC Executes)
  (A : Agent) (X : Action) (t : Nat) (d : AoC) (hd : inAoC X d)
  (g : Nat) (e : Grant Principal Agent Action) (t'' : Nat)
  (hres : obs.resolve A X t = some g) (hled : obs.ledger g = some e)
  (hw : e.made < t'' ∧ t'' ≤ t ∧ obs.withdrawn g t'') :
  ¬ Executes A X t := by
  intro hex
  obtain ⟨g', e', hres', hled', _, _, _, _, hst⟩ := h A X t d hd hex
  have hg : g = g' := Option.some.inj (hres.symm.trans hres')
  subst hg

```

```

have he : e = e' := Option.some.inj (hled.symm.trans hled')
subst he
exact hst ⟨t', hw⟩

```

```

end Defs

```

```

-- =====
-- Countermodels: each conjunct is separately load-bearing
-- against ThirdPartyAccountable. Unit types, execution at t = 1.
-- =====
section Countermodels

```

```

def ex1 : Unit → Unit → Nat → Prop := fun _ _ t => t = 1
def aocU : Unit → Unit → Prop := fun _ _ => True

```

```

def e0 : Grant Unit Unit Unit :=
{ principal := (), agent := (), names := fun _ => True, bound := fun _ => True, made := 0 }

```

```

/-- The reference model: everything holds. -/

```

```

def obsGood : Observable Unit Unit Unit :=
{ resolve := fun _ _ _ => some 0, ledger := fun _ => some e0,
  registered := fun _ => True, withdrawn := fun _ _ => False }

```

```

theorem good_accountable : ThirdPartyAccountable obsGood aocU ex1 := by
  intro A X t d _ ht
  cases A; cases X; cases d
  subst ht
  exact ⟨0, e0, rfl, rfl, trivial, rfl, trivial, trivial, Nat.zero_lt_one,
    fun ⟨_, _, _, hw⟩ => hw⟩

```

```

-- CM-C: no public binding. Appointment occurred; the third party cannot
-- reproduce the resolution. (Appointed holds, C fails.)

```

```

def obsC : Observable Unit Unit Unit := { obsGood with resolve := fun _ _ _ => none }

```

```

theorem cm_C :

```

```

  ¬ ThirdPartyAccountable obsC aocU ex1 ∧
  ¬ dC obsC () () () 1 ∧
  (dI obsC () ∧ dM obsC () () ∧ dS obsC () () ∧ dR obsC () () () 1 ∧ dApp obsC () () () 1) :=
  ⟨fun h => (let ⟨_, _, hres, _⟩ := h () () 1 () trivial rfl; nomatch hres),
  fun ⟨_, _, hres, _⟩ => (nomatch hres),
  trivial,
  ⟨0, e0, rfl, rfl, trivial⟩,
  ⟨0, e0, rfl, rfl, trivial⟩,
  fun _ _ hres _ _ => (nomatch hres),

```

⟨0, e0, rfl, rfl, rfl, trivial, trivial, Nat.zero_lt_one⟩⟩

-- CM-I: binding resolves to a principal no registry names (pseudonymous key).

def obsI : Observable Unit Unit Unit := { obsGood with registered := fun _ => False }

theorem cm_I :

¬ ThirdPartyAccountable obsI aocU ex1 ∧
 ¬ dI obsI () ∧
 (dC obsI () () () 1 ∧ dM obsI () () ∧ dS obsI () () ∧ dR obsI () () () 1 ∧ dApp obsI () () () 1) :=
 (fun h => (let ⟨_, _, _, _, hreg, _⟩ := h () () 1 () trivial rfl; hreg),
 fun h => h,
 ⟨0, e0, rfl, rfl, rfl, rfl⟩,
 ⟨0, e0, rfl, rfl, trivial⟩,
 ⟨0, e0, rfl, rfl, trivial⟩,
 fun g e hres hled _ => by
 have hg : 0 = g := Option.some.inj hres
 subst hg
 have he : e0 = e := Option.some.inj hled
 subst he
 exact fun ⟨_, _, _, hw⟩ => hw,
 ⟨0, e0, rfl, rfl, rfl, trivial, trivial, Nat.zero_lt_one⟩⟩

-- CM-M: the grant states a bound but names no act (blanket authority).

def eM : Grant Unit Unit Unit := { e0 with names := fun _ => False }

def obsM : Observable Unit Unit Unit := { obsGood with ledger := fun _ => some eM }

theorem cm_M :

¬ ThirdPartyAccountable obsM aocU ex1 ∧
 ¬ dM obsM () () ∧
 (dC obsM () () () 1 ∧ dI obsM () ∧ dS obsM () () ∧ dR obsM () () () 1) :=
 (fun h =>
 let ⟨_, e, _, hled, _, _, hn, _⟩ := h () () 1 () trivial rfl
 have he : eM = e := Option.some.inj hled
 (he ▶ hn : eM.names ()),
 fun ⟨_, e, hled, _, hn⟩ =>
 have he : eM = e := Option.some.inj hled
 (he ▶ hn : eM.names ()),
 ⟨0, eM, rfl, rfl, rfl, rfl⟩,
 trivial,
 ⟨0, eM, rfl, rfl, trivial⟩,
 fun g e hres hled _ => by
 have hg : 0 = g := Option.some.inj hres
 subst hg
 have he : eM = e := Option.some.inj hled

```
subst he
exact fun ⟨_, _, _, hw⟩ => hw
```

-- CM-S: the grant names the act but the act is outside its stated bound.

```
def eS : Grant Unit Unit Unit := { e0 with bound := fun _ => False }
def obsS : Observable Unit Unit Unit := { obsGood with ledger := fun _ => some eS }
```

theorem cm_S :

```
  ¬ ThirdPartyAccountable obsS aocU ex1 ∧
  ¬ dS obsS () () ∧
  (dC obsS () () () 1 ∧ dI obsS () ∧ dM obsS () () ∧ dR obsS () () () 1) :=
  ⟨fun h =>
    let ⟨_, e, _, hled, _, _, _, hb, _⟩ := h () () 1 () trivial rfl
    have he : eS = e := Option.some.inj hled
    (he ▶ hb : eS.bound ()),
  fun ⟨_, e, hled, _, hb⟩ =>
    have he : eS = e := Option.some.inj hled
    (he ▶ hb : eS.bound ()),
  ⟨0, eS, rfl, rfl, rfl, rfl⟩,
  trivial,
  ⟨0, eS, rfl, rfl, trivial⟩,
  fun g e hres hled _ => by
    have hg : 0 = g := Option.some.inj hres
    subst hg
    have he : eS = e := Option.some.inj hled
    subst he
    exact fun ⟨_, _, _, hw⟩ => hw
```

-- CM-R: the resolved grant was withdrawn at $t'' = 1$, at the execution moment.

```
def obsR : Observable Unit Unit Unit :=
  { obsGood with withdrawn := fun _ t'' => t'' = 1 }
```

theorem cm_R :

```
  ¬ ThirdPartyAccountable obsR aocU ex1 ∧
  ¬ dR obsR () () () 1 ∧
  (dC obsR () () () 1 ∧ dI obsR () ∧ dM obsR () () ∧ dS obsR () () ∧ dApp obsR () () () 1) :=
  ⟨fun h =>
    let ⟨_, e, _, hled, _, _, _, _, hst⟩ := h () () 1 () trivial rfl
    have he : e0 = e := Option.some.inj hled
    hst ⟨1, he ▶ Nat.zero_lt_one, Nat.le_refl 1, rfl⟩,
  fun h => h 0 e0 rfl rfl rfl ⟨1, Nat.zero_lt_one, Nat.le_refl 1, rfl⟩,
  ⟨0, e0, rfl, rfl, rfl, rfl⟩,
  trivial,
  ⟨0, e0, rfl, rfl, trivial⟩,
```

```

    ⟨0, e0, rfl, rfl, trivial⟩,
    ⟨0, e0, rfl, rfl, rfl, trivial, Nat.zero_lt_one⟩⟩

-- CM-App: the grant is made AT the execution moment, not before (post-hoc
-- entry). The binding resolves; there was no pre-execution appointment.
-- (C holds, Appointed fails.)
def eA : Grant Unit Unit Unit := { e0 with made := 1 }
def obsA : Observable Unit Unit Unit := { obsGood with ledger := fun _ => some eA }

theorem cm_App :
  ¬ ThirdPartyAccountable obsA aocU ex1 ∧
  ¬ dApp obsA () () () 1 ∧
  (dC obsA () () () 1 ∧ dI obsA () ∧ dM obsA () () ∧ dS obsA () () ∧ dR obsA () () () 1) :=
  (fun h =>
    let ⟨_, e, _, hled, _, _, _, hmade, _⟩ := h () () 1 () trivial rfl
    have he : eA = e := Option.some.inj hled
    Nat.lt_irrefl 1 (he ▸ hmade : eA.made < 1),
    fun ⟨_, e, hled, _, _, _, hmade⟩ =>
      have he : eA = e := Option.some.inj hled
      Nat.lt_irrefl 1 (he ▸ hmade : eA.made < 1),
    ⟨0, eA, rfl, rfl, rfl, rfl⟩,
    trivial,
    ⟨0, eA, rfl, rfl, trivial⟩,
    ⟨0, eA, rfl, rfl, trivial⟩,
    fun g e hres hled _ => by
      have hg : 0 = g := Option.some.inj hres
      subst hg
      have he : eA = e := Option.some.inj hled
      subst he
      exact fun ⟨_, _, _, hw⟩ => hw)

/-- Finding B, both directions, in one statement: C and Appointed diverge
in both orders at concrete instances of the derived predicates. -/
theorem C_and_Appointed_separate :
  (dApp obsC () () () 1 ∧ ¬ dC obsC () () () 1) ∧
  (dC obsA () () () 1 ∧ ¬ dApp obsA () () () 1) :=
  ⟨⟨cm_C.2.2.2.2.2, cm_C.2.1⟩, ⟨cm_App.2.2.1, cm_App.2.1⟩⟩

end Countermodels
end Synthetic

#print axioms Synthetic.synthetic_necessity
#print axioms Synthetic.authorized_implies_O
#print axioms Synthetic.dC_functional

```

```
#print axioms Synthetic.single_governor
#print axioms Synthetic.revocation_dominates
#print axioms Synthetic.good_accountable
#print axioms Synthetic.cm_C
#print axioms Synthetic.cm_I
#print axioms Synthetic.cm_M
#print axioms Synthetic.cm_S
#print axioms Synthetic.cm_R
#print axioms Synthetic.cm_App
#print axioms Synthetic.C_and_Appointed_separate
```